Database Tuning

- 这次实验的内容非常多，所以我想通过一道比较有典型意义的题目来说明调优的过程和思考。

实验活动3：综合调优：

- **查询1**：查找指定某一门课中具有相同成绩的学生，输出列：（学号，姓名，成绩），先按他们的成绩降序排列，再按学号排序。

调优时应注意的地方

- 需要在每次执行完一次查询后清空一次buffer，这样才能保证每次查询的结果能够进行比较。实验证明：若不清空buffer，第二次查询时间甚至可能只有第一次查询的1/10，这样将使查询时间无法进行正确比较
- 应该取多次查询的平均值作为某种查询的效率评价

第一种查询

```
select  sc1.sno, student.sname, sc1.grade
from student,sc sc1,sc sc2
where sc1.cno='86'
 and  sc2.cno='86'
 and  sc1.grade=sc2.grade
 and  sc1.sno<>sc2.sno
 and  student.sno=sc1.sno
order by sc1.grade desc,sc1.sno asc
```

第一种查询（续）

- 这个查询的执行时间（一遍）约为77秒（几次执行时间的平均结果,下面计算的时间均为平均结果），效率相当低！
- 没有去重，执行结果有340万个元组！对340万个元组进行ORDER BY操作，想不慢都难。

第二种查询

```
select distinct sc1.sno, student.sname,
        sc1.grade
from student,sc sc1,sc sc2
where sc1.cno='86'
 and  sc2.cno='86'
 and  sc1.grade=sc2.grade
 and  sc1.sno<>sc2.sno
 and  student.sno=sc1.sno
order by sc1.grade desc,sc1.sno asc
```

第二种查询（续）

- 这个查询的执行时间（一遍）约为19.3秒。效率有所提高，结果集为9400个元组！
- 在这个查询语句中，做了一个去重的步骤，这个步骤虽然需要对结果排序，但能够使的其后的ORDER BY代价大大减少

第二种查询（续）

- 但这仍不是一种好的查询！
- 通过观察它的执行计划图，可以发现此查询计划完全采用table scan的方式来查找符合where条件的元组，而且sc表中有一万个元组，sc表和sc表做join耗费了大量的I/O时间。这两个因素是造成查询效率低下的最主要原因。

第三种查询

```
select distinct sc1.sno, student.sname,sc1.grade
from student,sc sc1
where student.sno=sc1.sno
 and  sc1.cno='86'
 and  sc1.grade in
                (select grade
                 from sc sc2
                 where cno='86')
order by sc1.grade desc,sc1.sno asc
```

第三种查询（续）

- 此查询的执行时间约为1.43秒，比上一个查询的执行时间降低了一个数量级！！！
- 通过分析执行计划图，可以看出，虽然在sc表上查找cno=‘86’的元组依然用的是table scan的方式，但是嵌套子查询的结果大大缩小了要与sc1表进行join的表的大小，省去了大量I/O代价，所以执行效率有了非常明显的提高。
- 而且，这个查询语句中的嵌套子查询为非相关的，先执行子查询得到一个小表，这也使得效率比较高。

第四种查询

```
select distinct sc1.sno, student.sname, sc1.grade
from  student ,sc sc1
where student.sno=sc1.sno
and sc1.cno='86'
and exists
    (select *
    from sc sc2
    where cno='86'
    and sc1.sno<>sc2.sno
    and  sc1.grade=sc2.grade
)
order by sc1.grade desc, sc1.sno asc
```

第四种查询（续）

- 本来提出第4种查询是想要比较一下用exists量词的子查询和用in谓词的子查询的执行效率差异
- 但是实际的执行时间是1.37秒，并没有什么显著的差别。
- 因为在书上有这么一段话：“由于带exists量词的相关子查询只关心内层查询是否有返回值，并不需要查具体值，因此其效率并不一定低于不相关子查询，有时是高效的方法。”

第四种查询（续）

- 但很明显，在这道题里，这不是一种高效的方法
- 通过观察执行计划图发现，原来此查询的执行计划与上一种查询的完全一样。

第五种查询

```
select distinct sc1.sno, student.sname, sc1.grade
from  student ,sc sc1
where student.sno=sc1.sno
and sc1.cno='86'
and grade in
    (select sc2.grade
      from  sc sc2
      group by grade
     having count(*) > =2
    )
order by sc1.grade desc, sc1.sno asc
```

第五种查询（续）

- 此查询的执行时间约为1.93秒，还是没什么改进，甚至有些倒退
- 通过仔细观察它的执行计划图可以发现，经过对grade分组后选出的结果的确使得非相关嵌套子查询得到了一个更小的临时表，进一步降低了表连接的代价，可是与查询计划3未做分组的相比，节省的代价并不十分可观。而且这方面的节省，又被进行count(*)操作时对结果元组进行排序所增加的成本抹掉了，所以执行效率没有提高。

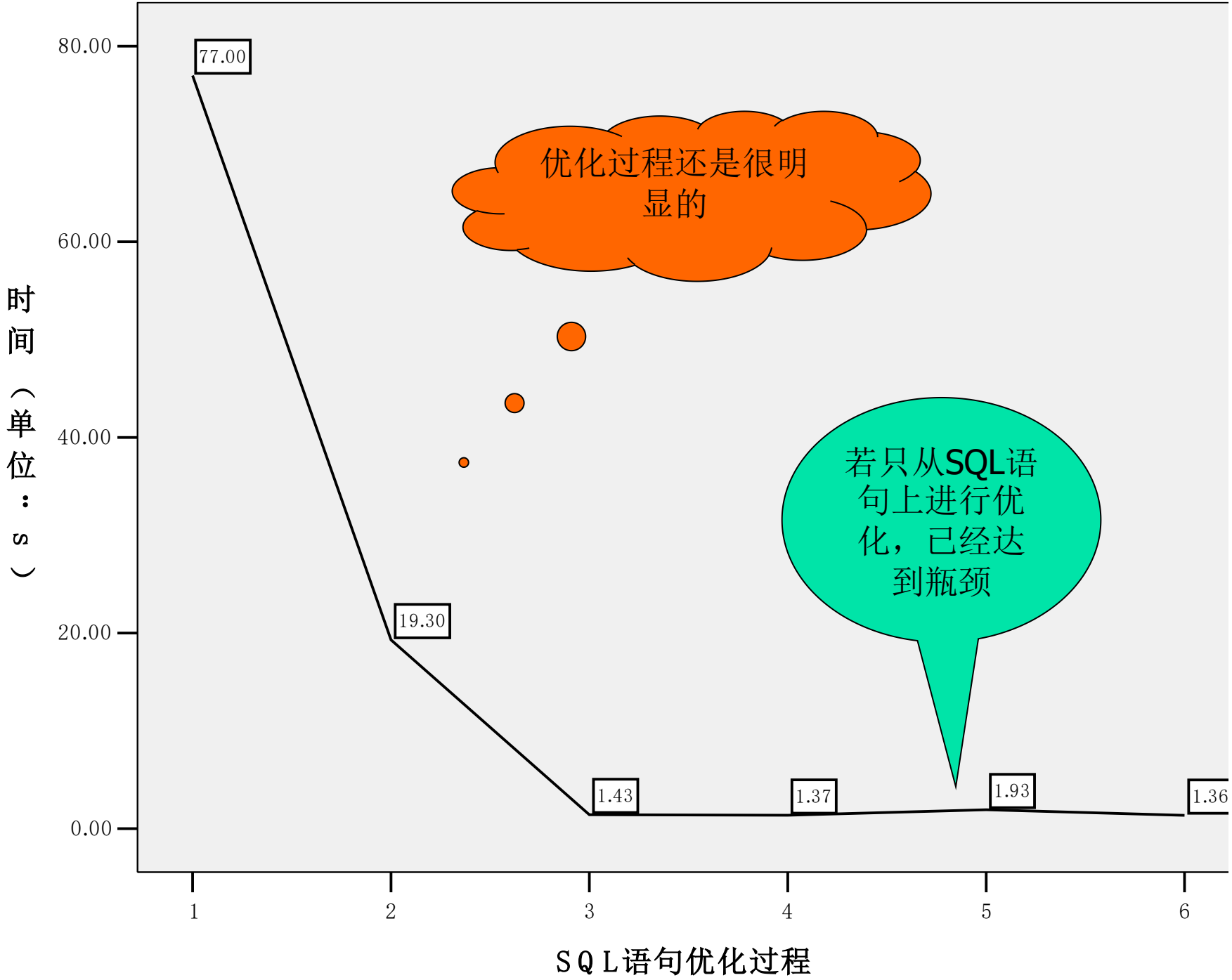
第六种查询

```
create view smallsc as
    select sc.sno,sc.grade
        from sc
        where sc.cno='86'

select distinct  smallsc.sno, student.sname, smallsc.grade
from  student,smallsc
where  student.sno=smallsc.sno
and  smallsc.grade in
                (select grade
                 from smallsc smallsc2
                 where smallsc.sno<>smallsc2.sno)
order by smallsc.grade desc,smallsc.sno asc
```

第六种查询（续）

- 此查询的执行时间是1.36秒，相差无几。
- 但是这个统计时间是在视图smallsc已经建立好的条件下，执行SQL语句的结果。未把建立视图的时间计入总的执行时间里。
- 这也说明了视图确实只是一个虚表，每次查询时，其实还是把生成视图的SQL语句再执行一遍


80.00
60.00
40.00
20.00
0.00
时间（单位：s）
77.00
19.30
1.43
1.37
1.93
1.36
1
2
3
4
5
6
SQL语句优化过程
优化过程还是很明显的
若只从SQL语句上进行优化，已经达到瓶颈

查询SQL语句上的改进总结

- 到此，可以发现，在SQL语句的改造上已经遇到了瓶颈，很难再有比较大的提高。
- 以上的改进都是在SQL语句上作修改，以期在逻辑层次上降低代价，提高效率。但是经过多种形式的SQL语句的测试，发现通过SQL语句的改进有一个阀值，很难突破。因此，更进一步的优化应该深入到物理层次上，这就涉及到了表的连接、扫描在系统内部的实现机制。建立索引就是在此方面改进的最好方式。

索引计划1

在sc(grade)和sc(cno)上分别建立普通索引

SQL语句仍为：

```
select distinct sc1.sno, student.sname,sc1.grade
from student,sc sc1
where student.sno=sc1.sno
 and  sc1.cno='86'
 and  sc1.grade in
                (select grade
                 from sc sc2
                 where cno='86'
                  and  sc1.sno<>sc2.sno)
order by sc1.grade desc,sc1.sno asc
```

索引计划1分析

- 加了索引后的执行时间仍为1.36秒！与未加索引时一样
- 通过看查询计划图得知，加了索引后的查询执行计划与没有索引的一样，仍采用表扫描。

索引计划2

- 在sc(grade)上建立聚簇索引，
- 执行时间为1.01秒。
- 多点查询情况下，建立聚簇索引效率不一定会提高，甚至可能更低，这点在前面第二部分的实验中已经证明

索引计划3

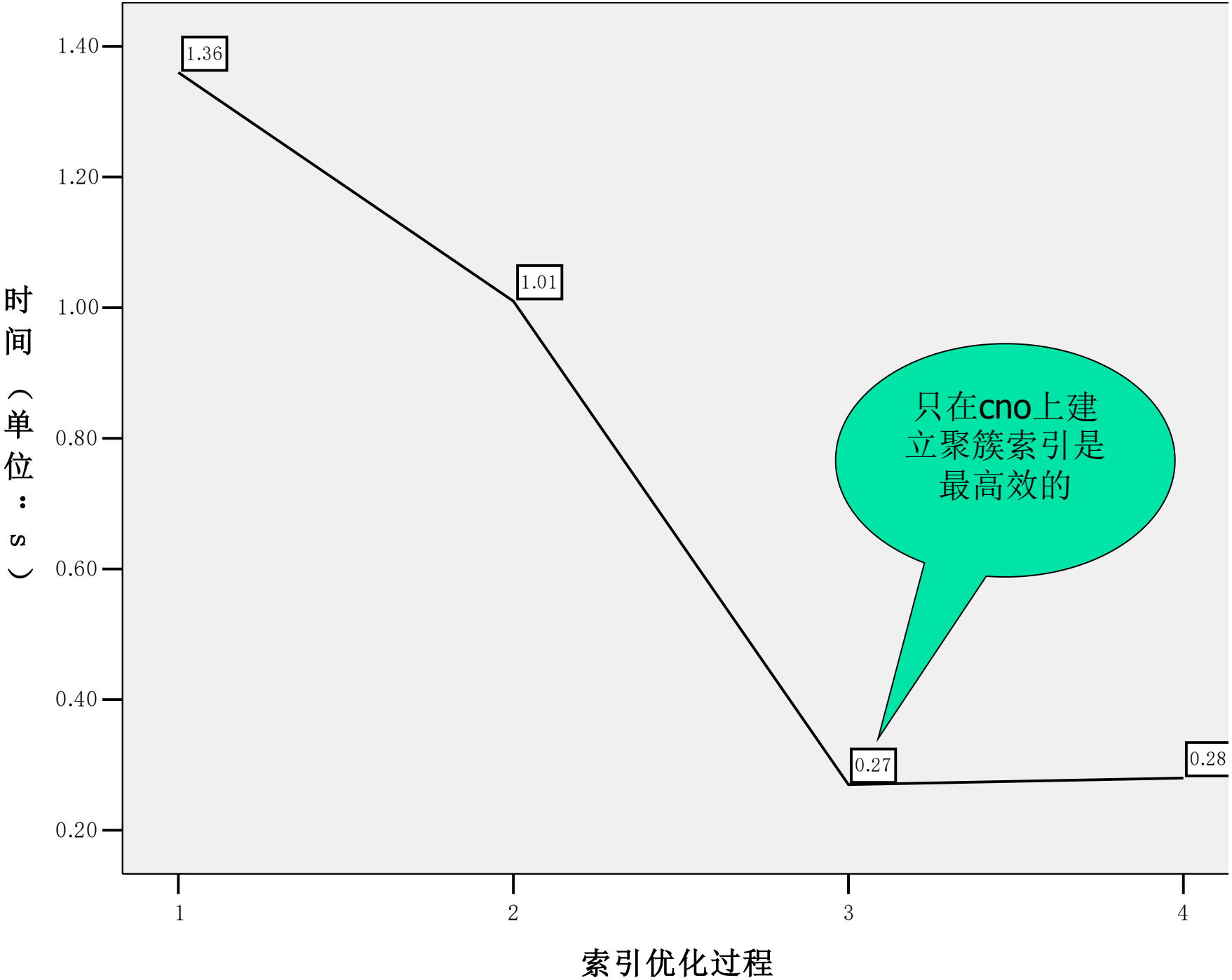
- 在sc(cno)上建立聚簇索引
- 执行时间为0.27秒
- 这个效率提高了许多，这是因为在点查询时，利用聚簇索引会提高查询速度。这也正是本次实验中第一部分实验的内容。

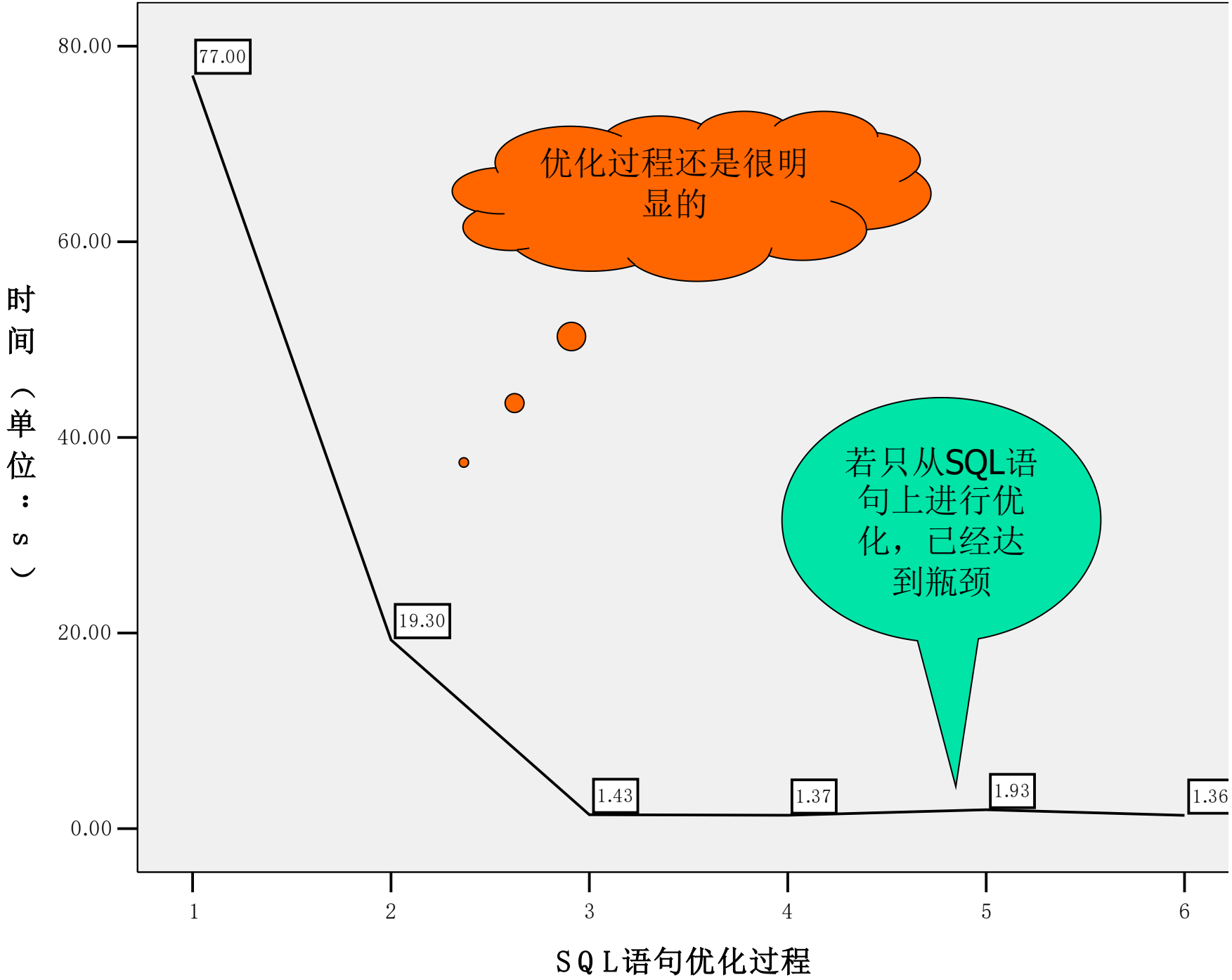
索引计划4

- 在sc (cno,sno, grade)上建立聚簇索引
- 执行时间是也为0.28秒，没有比只在sc(cno)上建聚簇索引有明显的改善

索引计划N

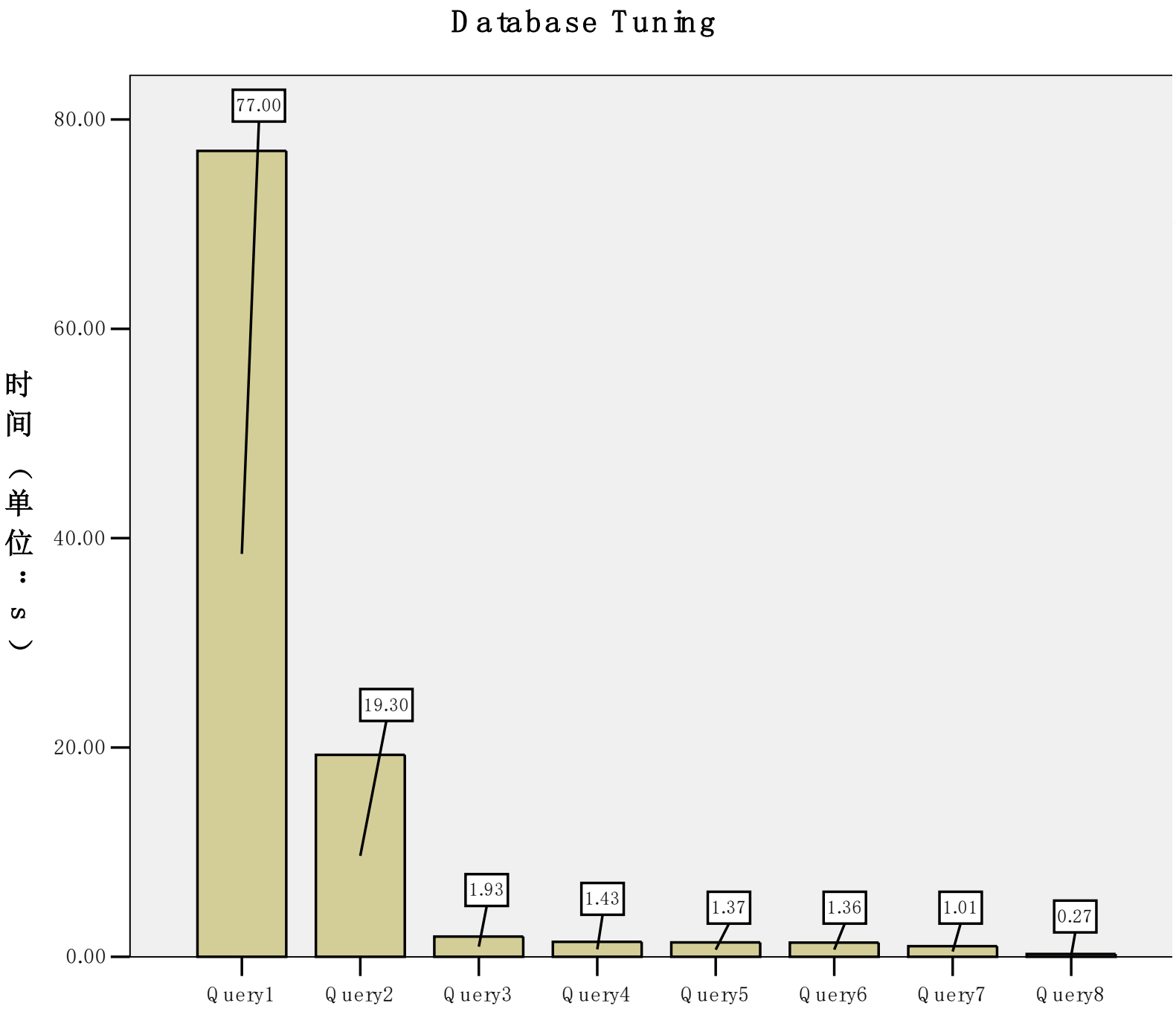
- 至于是否在student上加索引，由于从前面的执行计划中可以看见，student表的table scan只占总时间的3%，所以并不一定在其上建索引
- 在实际测试中甚至发现，在student上建索引还会降低查询速度。这可能是由于student表比较小的缘故，原本可以直接装进内存中处理


1.40
1.20
1.00
0.80
0.60
0.40
0.20
时间（单位：s）
1.36
1.01
0.27
0.28
只在cno上建立聚簇索引是最高效的
1
2
3
4
索引优化过程


80.00
60.00
40.00
20.00
0.00
时间（单位：s）
77.00
19.30
1.43
1.37
1.93
1.36
1
2
3
4
5
6
SQL语句优化过程
优化过程还是很明显的
若只从SQL语句上进行优化，已经达到瓶颈

建立索引的改进总结

- 在恰当的属性列上建立恰当的索引能够极大地提高执行效率
- 但是，索引建立不当就可能不会起到任何改进的作用，甚至还有可能降低效率
- 索引是有用的，但不总是有用的，所以建立索引要视具体情况而定


Database Tuning
时间（单位：s）
80.00
60.00
40.00
20.00
0.00
77.00
19.30
1.93
1.43
1.37
1.36
1.01
0.27
Query1
Query2
Query3
Query4
Query5
Query6
Query7
Query8

实验中遇到的问题总结

- 在使用工具exptool计算执行时间的时候，我发现执行相同的SQL查询语句，exptool计算的时间大不相同，而且第二次执行的时间和第一次执行时间往往相差一个数量级！
- 这是怎么回事呢？

问题一

- 经过思考，我认为这是因为第一次执行后我没有清空内存缓冲区，所以第一次的执行时间是真正从硬盘数据库中取出查询结果的时间，而第二次就是直接从内存中读出已有的结果，所以时间大大缩短了

问题二

- 经过实验证明，若执行10次的时间是10秒，那么执行一次的时间不是10秒/10=1秒！而往往要比1秒长很多。所以不能用执行多次简单求平均值的方法来确定实际一次的执行时间。
- 我认为原因大概和上题相同，是查询结果留在内存缓冲区中造成的，后面的查询比前面快。

问题三

- 即使每次执行完都清空缓冲区，查询的执行结果也会有差异，不是很稳定。
- 所以最好的办法就是测量几次，求其平均值

问题四

- 在实验过程中，我发现我与其他同学计算出来的时间相差较大，后来终于发现这是由于内存大小不同，主频不同等计算机硬件问题造成的，所以每个人的时间都不会完全一样。

实验体会

- 感觉虽然这次实验的工作量比较大，但是实验中值得深思，向下钻研的地方特别多，所以做起来也比较有意思。

实验体会（续）

- 对于能够获得同一个结果集的查询，不同的实现方法会使其查询效率大相径庭！
- 只有真正深刻理解了SQL语句转换成关系代数，在系统内部的逻辑和物理层次上究竟是如何实现查询的，才能做好调优。
- 所以调优是对综合能力的考验，它概括了我们所学过的大部分数据库知识。